

Recursion, Fractals and space-filling curves in Logo

Alina Musaeva

Julian Schwarzer

Lars Wittner

June 2026

Abstract

Abstract content [1].

1 Introduction

In chapter 1 we will explore what Logo is, its main features and how to work with it. We will show you some base commands which are required to better understand it in other chapters. In chapter 2, we will methodically explore the concepts of recursion and fractals using the Pythagoras tree, Koch's flake. Finally, we will cover topics such as L-System, edge- and node-rewriting and branching. This will enable you to understand the idea of space-filling curves in Logo.

1.1 Definition of Logo and its origin

Before we move on to recursion and fractals, we're going to find out what Logo is and learn some base commands.

Logo is a programming language that is well-known for its use of turtle graphics. With Logo, you can draw different figure shapes.

It was invented in 1966 in Cambridge (USA) by Wally Feurzeug, Seymour Papert, Daniel Bobrow and Cynthia Solomon.

The original intention behind the creation of this language was to provide a means for children in schools to learn how to code and enhance their mathematical skills.

1.2 Turtle Graphics

The turtle is a pen or a cursor (often represented as a triangle) that moves after a command and draws when the pen is down. You can change the position, direction, pen state, color etc. It will all be displayed on the computer screen.

1.3 Data Types

In Logo, data types are dynamic. This means that they do not need to be defined explicitly.

There are four main data types: word, number, list and boolean.

1. *word* is used for a sequence of characters
2. *number* for such numbers as integer, float, double etc.
3. *list* is a list of numbers or characters (which may for example be a sentence or an array).
4. *bool* is for true or false



Figure 1: command forward and right in ucbLogo

1.4 Base commands

The Logo syntax is very intuitive. It consists of simple words, with or without parameters (1).

A shortcut is also available for each command, thus avoiding the need to type the entire word each time.

Here are the base commands:

```
1 FORWARD/BACK steps
2 RIGHT/LEFT degree
3 CLEARSCREEN
4 PENUP/PENDOWN
5 REPEAT
6 PRINT [The sentence]
```

logo

As demonstrated in figure 1, the turtle was moved forward 40 steps, rotated 110 degrees, and moved forward again 49 steps.

The others commands for instance CLEARSCREEN or simply CS is used for cleaning the drawings on the display (not the code).

PENUP is used to move the turtles position without drawing and PENDOWN to return to the drawing mode.

REPEAT allows commands to be repeated multiple times, thus saving time.

For example we can draw a square using following command:

```
1 REPEAT 4 [fd 90 rt 90]
```

logo

1.5 Procedures

The following steps are required to create a procedure (function) in Logo:

Enter 'To' followed by the name of the procedure and 'End'. The commands should then be entered between these two items. Use parameters if required.

```
1 To ProcedureName :parameter1 ... parameter 2
2   [commands]
3 End
```

logo

1.6 Loops

If/Ifelse

```
1 IFELSE condition [condition true][condition false]
2 IF condition [condition true]
```

logo

While-loop

```
1 WHILE [condition]
2 [commands]
```

logo

For-loop

```
1 FOR [var start end step]
2 [commands]
```

logo

2 Recursion

We are now capable of writing programs with these base commands but we still have to introduce recursion for a functional programming language like Logo.

2.1 Concept

Recursion has its origin in the latin word *recurrere* (*to go back*). A recursive written function has to contain three key parts:

1. Base case
2. Calling itself
3. Recursive step

The base case will be a condition that indicates on what case our recursive calls will end up terminating. We need this due to our limitations of storage and to avoid an infinite amount of calls on our call stack. To actually reach this base case we need a recursive step that needs to be provided by our recursive call that approaches our base case.

Fibonacci numbers

```
1 private static int fib(int n) {
2     if (n <= 0) return -1; // wrong input
3     if (n == 1) return 0; // base case 1
4     if (n == 2) return 1; // base case 2
5     return fib(n - 1) + fib(n - 2); // recursive call
6 }
```

java

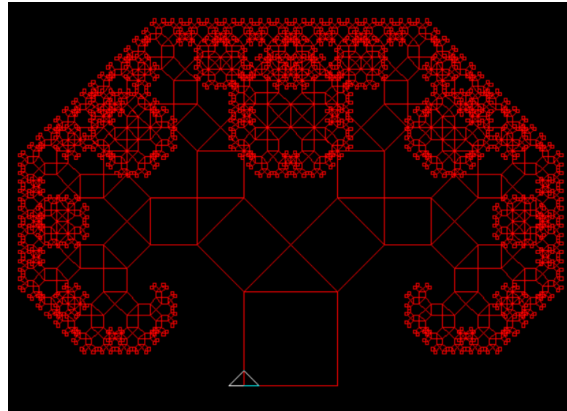


Figure 2: Pythagoras tree

In this java implementation of the Fibonacci numbers you can see that there are also multiple base cases possible because the first Fibonacci number has to be 0 and the second one needs to be 1. After these base cases the recursive call with the recursive steps are embedded in the returned value of this method. As you might notice there might be a problem by this implementation.

Very important is that the recursive step actually reaches our base case. In this example each reduction ($n - 1$ and $n - 2$) will either reach our first or our second base case and terminate. If we would have skipped our base case and not had filtered out our wrong inputs ($n \leq 0$) we would not terminate and the recursion will end up in an infinite loop.

The phrase *depth of recursion* determines the amount of nested recursive functions inside of itself.

Recursion often finds usage in science like mathematics or computer science *eg. factorials, Fibonacci*.

2.2 Example: Pythagoras tree

The pythagoras tree is a very common example for explaining recursion. Its main shape is a square with recursively placed squares on top of the base square rotated by a predefined angular that calculates based on the famous Pythagoras formula (2).

To implement this we will use the previously explained scheme: We declare our procedure within the first line (keyword *to*) with its name being *pythagoras* and two needed parameters (indicated by the colons) as *length* and *depth* and end our definition with the keyword *end* in our last line.

```
1  to pythagoras :length :depth
2    ...
3  end
```

logo

After this we have our main form drawn (the square) which will be drawn by repeating four times going forward and then turning right.

```
1  repeat 4 [fd :length rt 90]
```

logo

Now we write down our base case what translates to our program to end the procedure if we reach a depth of 0. The depth represents the maximum amount of squares placed on top of another square (recursion depth).

```
1  if :depth = 0 [stop]
logo
```

We and up at our starting position at the left bottom corner of the just drawn square. To draw our recursive squares we need to correct our position. For that we are going forward by our length and continue by rotating to the left by 45 degree which will be our predefined rotation so both squares will have the exact same length and our Pythagoras tree becomes axially symmetric.

```
1  fd :length lt 45
logo
```

Our recursive squares are then drawn with a reduced length in the scale of the Pythagoras formula and a depth that decreases by 1 which ensures the base case of depth 0 will be reached at some point.

```
1  pythagoras :length / (sqrt 2) :depth - 1
logo
```

Because we use an angle of 45 degree the following derivation explains the new length: We assume our hypotenuse as a and our two catheti as length b . According to Pythagoras:

$$\begin{aligned} a^2 + a^2 &= b^2 \\ 2a^2 &= b^2 \\ \sqrt{2}a &= b \\ b &= \frac{a}{\sqrt{2}} \end{aligned}$$

Now we need to correct our position again to resume with the second square.

```
1  rt 90 fd :length / (sqrt 2)
logo
```

Now at the right starting point for our last square, we perform the same actions as our first square.

```
1  pythagoras :length / (sqrt 2) :depth - 1
logo
```

After each recursive depth we return to the position of the previous one ($depth + 1$).

```
1  bk :length / (sqrt 2) lt 45 bk :length
logo
```

Our resulting procedure will result in the following:

Pythagoras tree

```
1  to pythagoras :length :depth ; variable length and depth
2  repeat 4 [fd :length rt 90] ; square
3  if :depth = 0 [stop] ; base case
4  fd :length lt 45 ; change pos
5  pythagoras :length / (sqrt 2) :depth - 1 ; recursive left
6  rt 90 fd :length / (sqrt 2) ; change pos
7  pythagoras :length / (sqrt 2) :depth - 1 ; recursive right
8  bk :length / (sqrt 2) lt 45 bk :length ; go back
9  end
logo
```

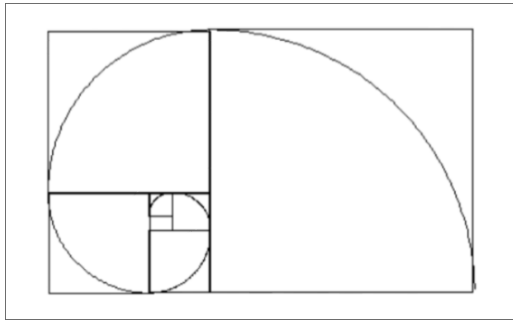


Figure 3: Golden ratio

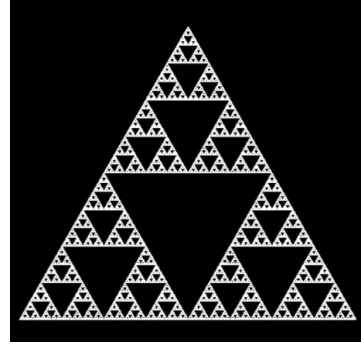


Figure 4: Sierpiński triangle

3 Fractals

3.1 Concept

Fractals are the idea of performing a recursion with a *recursion depth* of infinity. Like recursion, fractals also have their origin in a latin word (*fracere (to break)*).

Fractals are a theoretical concept that cannot be used in reality again due to eg. the finite resolution a monitor has.

The main property of fractals are the *self-similarity* which means you can identify the fractal itself as a part of itself by imagining to zoom infinitely in.

In order to be able to work with fractals nonetheless we can use visualisation by recursion with a relatively high recursion depth a . The depth of recursion is the scale of sharpness a fractal image has.

3.2 Example: Koch flake

The Koch flake became quite popular for its simplistic form and minimal lines of code for a quite nice looking shape of a snowflake. First described by Helge von Koch in 1904 is made up of three triangular shaped Koch curves. These Koch curves follow simple rules. At first a Koch curve simply is a line with a length. By increasing the depth there will be placed Koch flakes on top of this line forming a triangle on top of the line. The higher the depth the more triangles are placed on top of each other.

We will declare our Koch curve in the same way as we did with the Pythagoras tree.

```
1  to curve :length :depth
2    ...
3  end
```

logo

We will start by defining our termination condition (our *base case*) as drawing a *length*-long straight line.

Part 1/4:

```
1  ; base case: if depth is 0, move forward and stop
2  if :depth = 0 [fd :length stop]
```

logo

For achieving our Koch curve with a triangular shape placed on a line, our Koch curve consists of four parts: Now we write down our base case what translates to our program to end the procedure if we reach a depth of 0. The depth represents the maximum amount of squares placed on top of another square (recursion depth).

```

1  ; fractal part 1/4 (move)
2  curve :length / 3 :depth - 1
3  ; fractal part 2/4 (turn left and move)
4  lt 60
5  curve :length / 3 :depth - 1
6  ; fractal part 3/4 (turn right and move)
7  rt 120
8  curve :length / 3 :depth - 1
9  ; fractal part 4/4 (turn left and move)
10 lt 60
11 curve :length / 3 :depth - 1

```

logo

So now we have our Koch curve to get the shape of a snowflake we simply need to expend it by another procedure *flake* that repeats our *curve* three times and between each curve it rotates the turtle so we have a shape of a triangle.

Kock flake

```

1  ; procedure with parameters length and depth
2  to curve :length :depth
3  ; base case: if depth is 0, move forward and stop
4  if :depth = 0 [fd :length stop]
5  ; fractal part 1/4 (move)
6  curve :length / 3 :depth - 1
7  ; fractal part 2/4 (turn left and move)
8  lt 60
9  curve :length / 3 :depth - 1
10 ; fractal part 3/4 (turn right and move)
11 rt 120
12 curve :length / 3 :depth - 1
13 ; fractal part 4/4 (turn left and move)
14 lt 60
15 curve :length / 3 :depth - 1
16 end
17
18 to flake :depth
19 ; 3 sides of base triangle form
20 repeat 3 [
21   curve 200 :depth
22   rt 120 ; turn right to next triangle base side
23 ]
24 end

```

logo

4 Lindenmayer-System

The Lindenmayer-System or short the L-system was invented by Aristid Lindenmayer in 1968. He was a Hungarian Biologist who wanted to recreate plant growth as accurate as

$$\begin{aligned}
a &\rightarrow ab \\
ab &\rightarrow abbb \\
abbb &\rightarrow abbbbbbb
\end{aligned}$$

Figure 5: Example of words in a deterministic and context-free L-system.

possible and for this purpose he came up with this (V, ω, P) . V is the Alphabet which consists of non-terminal symbols and terminal symbols. Non-terminal symbols are symbols that will be replaced eventually, whereas terminal symbols will not be replaced at all. ω is a non-empty starting word of an L-system which will be used to form new words that will be part of the L-system. These words are created with Production rules P .

4.1 rewriting methods

This paper will only discuss the deterministic and context-free L-system. We start with an example to get the basic concept of those systems. When looking at $(\{a, b\}, a, P)$ where the production rule P describes $a \rightarrow ab$ which means whenever a word contains the letter a it is replaced in the next step for ab and $b \rightarrow bb$ which means every b will be replaced for bb . We start with the letter a because our starting word $\omega = a$ in our system a is replaced for ab . now we have two letters in our word and each of them will be replaced and they are replaced simultaneously. This is a big difference compared to normal context-free languages where only one symbol is replaced at a time. ab becomes $abbb$ because the a becomes ab and the b turns to bb , next $abbb$ turns to $abbbbbbb$ (Figure 5).

We will discuss two ways to create an L-System using the turtle-graphics. They are edge-rewriting and node-rewriting. In these rewriting systems the alphabet $V = \{+, -, F\}$. These characters form words that can be interpreted in the turtle graphics. $+$ means turning the head of the turtle to the right and $-$ means turning to the left, both with a predefined angle that does not change in one system. and F stands for going forward. The production rules in edge-rewriting replace the F , the edges for a new shape. The edges in node-rewriting do not get replaced, there are new symbols $A \notin F$ that have no interpretation in the turtle-graphic. Their only purpose is to be replaced.

4.1.1 Edge rewriting

Edge-rewriting is the more intuitive way of creating L-systems. For example, Figure 6 shows an L-system $(\{+, -, F\}, F, P)$ with $P := F \rightarrow +F - F$ and an angle $\delta = 45^\circ$. As shown earlier, L-systems have terminal symbols and non-terminal symbols. In our example $\{+, -\}$ are terminal symbols, they do not get replaced by our production rule and F is a non-terminal symbol since we have a rule that replaces the F for something else. However our F already had a meaning, namely being an edge in our L-system and so our edge is replaced for a shape as you can see in Figure 6. There is also the possibility to use two F , for example having now F_r and F_l for more variation and complexity in our L-system. So now we define $P := F_r \rightarrow +F_r - F_l, F_l \rightarrow -F_r + F_l$ and we start with the starting word $\omega = F_r$. The first rule is the same rule as in the earlier example and our second rule is the same shape to replace our lines but mirrored to the first rule. As seen in Figure 6, with two reproduction rules, the figure looks more complex.

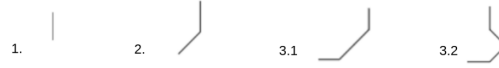


Figure 6: Example of L-system using edge-rewriting. 1. is before any rule was applied, 2. has a depth of 1. and 3.1 and 3.2 each have a depth of 2. 3.1 was created with one production rule, 3.2 was created with 2 production rules.

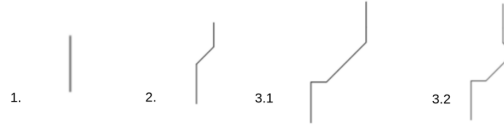


Figure 7: Example of L-system using node-rewriting. 1. is before any rule was applied, 2. has a depth of 1. and 3.1 and 3.2 each have a depth of 2. 3.1 was created with one production rule, 3.2 was created with 2 production rules.

4.1.2 Node rewriting

For Node-rewriting we do not replace our edges F anymore. We introduce new symbols $\{A, B, C, \dots\}$ that do not have any meaning in our system already. We will only use the symbol A in our example (Figure 7). Our new L-system $(\{+, -, F, A\}, FA, P)$ with $P := A \rightarrow +AF - AF$ and the angle $\delta = 45^\circ$. Since the non-terminal symbols do not have any meaning, they do not change the image that we get by illustrating our L-system, but they determine how the next step of the recursion will look like. As you can see in Figure 7, the edges that have been drawn in those with less depth, they are also in those with higher depth with new shapes in between them. Again we can create L-systems with multiple rules. We define $P := A_r \rightarrow +A_rF - A_lF$, $A_l \rightarrow -A_rF + A_lF$ and we choose $\omega = FA_r$. Just like with edge-rewriting the first two recursion steps look similar to the L-system with only one rule but with further depth the difference is clear.

4.2 Designs

The next step for making our fractals look more fabulous is to use designs. Designs are basically an ending repetition of our fractal. Just like we have seen earlier with the koch flake that we have created by repeating the koch curve three times. The first step is to choose any polygon as base for the rewriting. In our example (Figure 8) we will choose a square. With our square, we now have four edges already as base whereas we had only one in our fractal, for edge-rewriting instead of replacing our only line, we now have four lines, each being replaced by our shape (Figure 8 left side). We can do the same for node rewriting by placing nodes on the edges that every corner has a node and we replace those nodes for our shape. every time we replace our edges or nodes in any polygon, the line stops there, where it had begun (Figure 8)

4.3 Branching

Using edge/node- rewriting, we can now create many L-systems. By using the turtle graphics, we can draw any number of angles and lines like the Pythagoras tree, the Koch flake or any self made L-system lines or designs. But they will always just be a line, however the world of plants is nothing but a world of branching. Lindenmayer was a

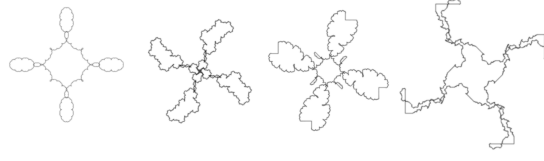


Figure 8: Designs created with a square, the left two designs are examples with edge-rewriting, left with one production rule, right with two production rules. the right two designs are examples with node-rewriting, left with one production rule, right with two production rules. All with a depth of 10.



Figure 9: Example of branching with edge-rewriting. Number 1 has depth=0, 2.has depth=1 and Number 3. has depth=2

Biologist and he wanted to recreate the world of plants. This too is possible to be made both with edge- and node-rewriting but we will only discuss it with edge-rewriting, but it works the same way with node-rewriting.

4.3.1 Stack

First we need to know what a stack is and how it works for us to use branching. A stack is a data structure to save data. Two functions are used to operate the stack. The first function is `push(input)`. When calling it, input will be written in our stack. The second function is `pop()`. It does not get any parameters and returns the last thing that was written on the stack removing it at the same time from it. So the stack uses the principle of Last in First out or "LiFo".

4.3.2 edge-rewriting with stack

We will introduce two new symbols in our alphabet. We define $[:= \text{push}((\text{position}, \text{direction}))$ and $] := \text{pop}()$. Every time that we will use $[$ in a word of our system we save the current position and the direction the turtle is facing and every time we read $]$ in our word the turtle returns to that position and the direction faced when we pushed that position. In our Example we used the L-system $(\{-, +, F, [,]\}, F, P)$ with $P := F \rightarrow F[-F][+F]F$ and $\delta = 45^\circ$. In Figure 9 we can see how we already have three branches with a depth of 1 in the 2. image. At the 3. image we can see why the stack is important, and why we could not use a list with "FiFo". The word for our 3. image is $F[-F][+F]F[-F[-F][+F]F][+F[-F][+F]F]F[-F][+F]F$ and when we interleave the braces like we did in that word we do not wanna mix up the order, because if we did, the image would already consist of three parts in our example which would not be a fractal anymore.

5 Conclusion

References

- [1] Juraj Hromkovic. *Einführung in die Programmierung mit LOGO*. Vol. 206. Springer, 2010.